

Shared Memory Cuda for Converting RGB image to Greyscale

Asked 8 years, 9 months ago · Modified 8 years, 9 months ago · Viewed 1k times

I am new to Cuda programming. I have a code that converts an RGB image to Greyscale. The algorithm for reading RGB values of pixel and converting them to GreyScale has been provided to us. Parallelizing the code has given me around 40-50x speed up. I want to optimize it further to achieve around 100x speedup. For this purpose I want to use shared memory access as its magnitude faster than Global Memory Access. I have gone through different online resources and have the basic understanding of shared memory access. But in my code I am having problem understanding how to implement shared memory. The code to read RGB values and converting to Greyscale

```
for ( int y = 0; y < height; y++) {
    for ( int x = 0; x < width; x++) {
        float grayPix = 0.0f;
        float r = static_cast< float >(inputImage[(y * width) + x]);
        float g = static_cast< float >(inputImage[(width * height) + (y * width) + x]);
        float b = static_cast< float >(inputImage[(2 * width * height) + (y * width) + x]);

        grayPix = ((0.3f * r) + (0.59f * g) + (0.11f * b));
        grayPix = (grayPix * 0.6f) + 0.5f;
        dstGrayImage[(y * width) + x] = static_cast< unsigned char >(grayPix);
    }
}
```

Input image a char* and we are using Cimg library to manipulate image

```
CImg< unsigned char > inputImage = CImg< unsigned char >(argv[1]);
```

Where user passes the path to image as a argument while running the code

This is my Cuda implementation of it

```
unsigned int x = (blockDim.x * blockDim.x) + threadIdx.x;
unsigned int y = (blockDim.y * blockDim.y) + threadIdx.y;
float grayPix = 0.0f;
float r = static_cast< float >(inputImage[(y * height) + x]);
float g = static_cast< float >(inputImage[(width * height) + (y * height) + x]);
float b = static_cast< float >(inputImage[(2 * width * height) + (y * height) + x]);
grayPix = ((0.3f * r) + (0.59f * g) + (0.11f * b));
grayPix = (grayPix * 0.6f) + 0.5f;
dstGrayImage[(y * height) + x] = static_cast< unsigned char >(grayPix);
```

The Grid and block and calling the code

```
dim3 gridSize(width/16, height/16);
dim3 blockSize(16,16);
gridSize.x *= gridSize.y;
gridSize.x *= gridSize.y;
gridSize.x *= gridSize.y;
gridSize.x *= gridSize.y;
```

where width and height are the width and height of input image. I tried with block size of (32,32) but it slowed down the code instead of speeding it up

Now I want to add shared memory but the problem the access to the input variable InputImage is quite non linear, so what values do I add to the shared memory I tried something like

```
unsigned int x = (blockDim.x * blockDim.x) + threadIdx.x;
unsigned int y = (blockDim.y * blockDim.y) + threadIdx.y;
extern __shared__ int s[];
s[x] = inputImage[x];
__syncthreads();
```

and then replacing inputImage with s in the implementation but that just gave a wrong output (all black image) Can you help me out here to understand how can I implement shared memory, if even its possible and useful and is there a way I can make my access in a more coalesced way?

Any help would be grateful

C++ image memory cuda Edit tags

Share Edit Follow Close Flag

edited Nov 9, 2015 at 14:12

asked Nov 9, 2015 at 13:57

 bob honest for this site

 Hassan Jall

12.2k 4 31 53

1,184 4 15 34

1 Answer

Sorted by: Date modified (newest first)

This can't work for several reasons:

```
2 unsigned int x = (blockDim.x * blockDim.x) + threadIdx.x;
extern __shared__ int s[];
s[x] = inputImage[x];
```

One reason is that we cannot use a global index (`x`) as a shared memory index, unless the data set is small enough to fit in shared memory. For an image of reasonably large dimensions, you cannot fit the entire image into a single instance of shared memory. Furthermore, you are using only one dimensional index (`x`) of a two dimensional data set, so this can't possibly make sense.

This suggests a general lack of understanding of how to use shared memory in a program. However, rather than trying to sort this out, we can observe that for a properly written RGB->grayscale code, shared memory usage is unlikely to provide any benefit.

Shared memory bandwidth benefits (which is what you are referring to when you say "magnitude faster") are valuable when there is *data* re-use. An RGB->grayscale code should not require any data re-use. You load each RGB quantity exactly once from global memory, and you store the computed grayscale quantity exactly once in global memory. Moving some of this data temporarily to shared memory is not going to speed anything up. You still have to do the global loads and global stores, and for a properly written code, this should be all that is necessary.

However in your question you've already suggested a possible improvement path: *coalesced* access. If you were to profile your posted code, you would find completely uncoalesced access patterns. For good coalescing, we want compound index calculations to have the property that the `threadIdx.x` variable is not multiplied by anything:

```
unsigned int y = (blockDim.x * blockDim.x) + threadIdx.x;
unsigned int x = (blockDim.y * blockDim.y) + threadIdx.y;
float grayPix = 0.0f;
float r = static_cast< float >(inputImage[(y * height) + x]);
|
|
|
y depends on threadIdx.x
```

But in your case, your index calculation is multiplying `threadIdx.x` by `height`. This will result in non-coalesced access. Adjacent threads in a warp will have varying `threadIdx.x`, and we want index calculations of adjacent threads in the warp to result in adjacent locations in memory, for good coalesced access. You cannot achieve this if you multiply `threadIdx.x` by anything.

The solution for this problem is quite simple. You should just use kernel code that is almost an exact duplicate of the non-CUDA code you have shown, with appropriate definitions for `x` and `y`:

```
unsigned int x = (blockDim.x * blockDim.x) + threadIdx.x;
unsigned int y = (blockDim.y * blockDim.y) + threadIdx.y;
if ((x == width) && (y == height)) {
    float grayPix = 0.0f;
    float r = static_cast< float >(inputImage[(y * width) + x]);
    float g = static_cast< float >(inputImage[(width * height) + (y * width) + x]);
    float b = static_cast< float >(inputImage[(2 * width * height) + (y * width) + x]);

    grayPix = ((0.3f * r) + (0.59f * g) + (0.11f * b));
    grayPix = (grayPix * 0.6f) + 0.5f;
    dstGrayImage[(y * width) + x] = static_cast< unsigned char >(grayPix);
}
```

Naturally, this is not a complete code. You have not shown a complete code, so if you respond with "I tried this but it doesn't work", it's unlikely I'll be able to help you much, since I don't know what code you're actually running. But:

1. Shared memory is not the right way to go for this algorithm
2. You undoubtedly have a coalescing issue in your posted code, for the reasons I indicate
3. The coalescing fix should follow the path I outlined
4. Your performance should improve with the coalescing fix.

Note that a response of "it doesn't work" means you are really asking for debugging assistance, not conceptual explanation, in which case you are **supposed** to provide an **MCVE**. What you have shown is not an MCVE. Preferably your MCVE should not depend on an external library like Cimg, which means it requires effort on your part to create one that would be a standalone test, but demonstrating the problem you are having.

Also, I would suggest whenever you are having trouble with a CUDA code, to use **proper CUDA error checking** as well as run your code with `cuda-memcheck`.

[Proper CUDA error checking would have identified a problem with your attempt to use shared memory, for example, due to out-of-bounds indexing in shared memory.]

Share Edit Follow Flag

edited May 23, 2017 at 11:44

answered Nov 9, 2015 at 14:57

 Community

 Robert Crovella

1 1

150k 12 239 284

¹ Thank You for the detailed answer and explaining the concepts. I will try implementing this and will post about the results. Your answer was quite detailed and informative and will help me understanding CUDA better – Hassan Jall Nov 9, 2015 at 16:24

² Oh I just made this changes and ran the code and I am getting speedups from 80-110x. Thank You, I cant believe the mistake was so simple, again thanks for the detailed answer, I not only got what was wrong but also a better understanding of coalescing, have a great day – Hassan Jall Nov 9, 2015 at 16:52